



Course Name		
Computer Programming Lab		
Lab Title		
Implement Iteration/ looping Programs in Python using WHILE and FOR loop		
Name	Registration Number	Lab #
Muhammad Hamzah Iqbal	F24604018	3
Date	Instructor's Name	Instructor's Signature
13/3/2024	LE Ayesha Ashfaque	

Objective:

- To have basic understanding of Loops in Python.
- To understand iteration of statements using for and while loop.

Theoretical Aspect

- **Looping in Programming**

Loops allow repeated execution of a block of code based on a condition or a sequence.

Python supports for and while loops to handle different iteration scenarios.

- **While Loop**

A while loop continues executing as long as the given condition remains True. It is useful when the number of iterations is not predetermined.

- **For Loop**

The for loop is used to iterate over sequences like lists, tuples, and strings. It is ideal when the number of iterations is known in advance.

- **Loop Control Statements**

Python provides break (to exit a loop early) and continue (to skip an iteration) to modify the behavior of loops and improve control over execution.

Experimental Procedure

Task 01:

1. Write a program that sums up a sequence of integers. Assume that the first integer read with the input function specifies the number of values remaining to be entered. Your program should read only one value with each input function call. A typical input sequence might be the following where the 5 indicates that the subsequent five values are to be summed. For example,

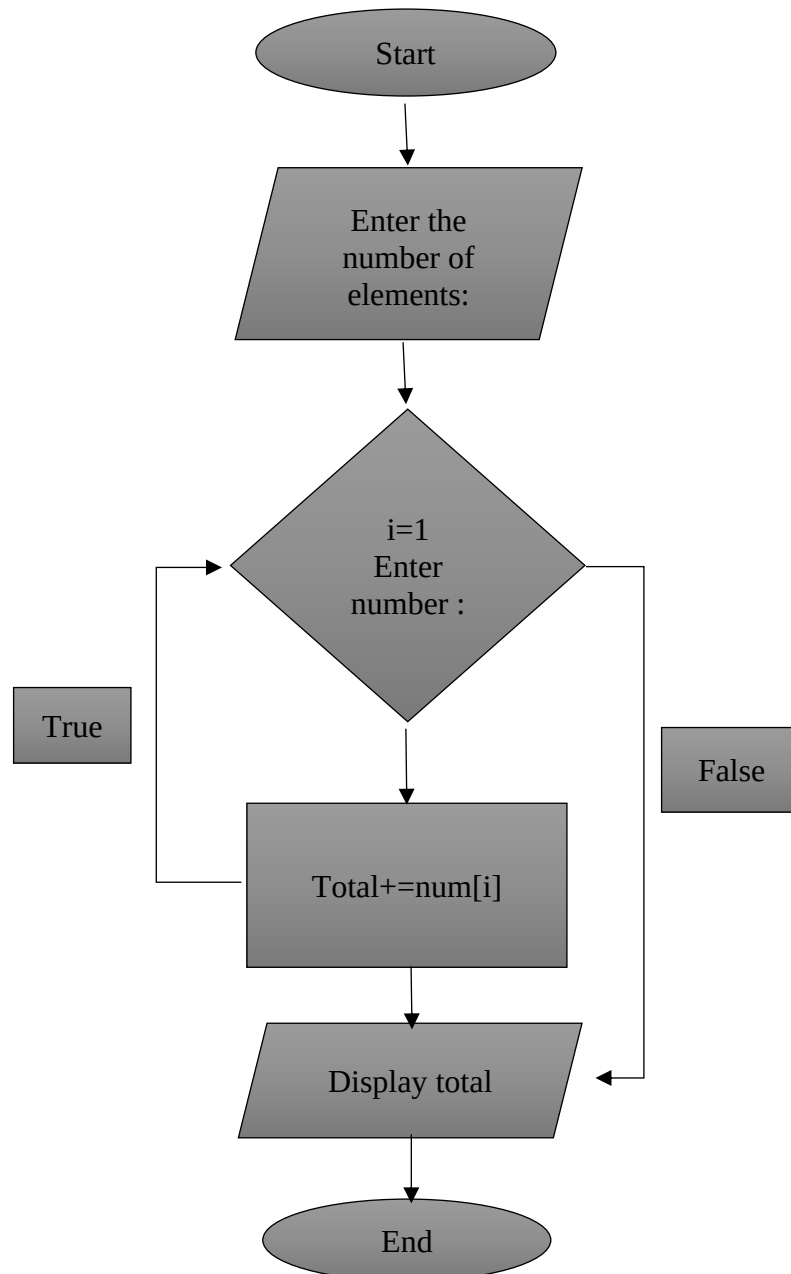
5
100
200
300
400
500.

Solution:

Algorithm:

- Start
- Input: Read the first integer n (the number of values to be summed).
- Initialize: Set total to 0.
- Loop: For i from 1 to n:
 - a. Input: Read the next integer value.
 - b. Update: Add value to total (i.e., $\text{total} = \text{total} + \text{value}$).
- Output: Print total.
- End

Flow Chart:



Experimental result:

```
1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5
6 n=int(input())
7 total=0
8 for i in range (n):
9     num=int(input())
10    total+=num
11
12 print(total)
```

Figure 2 Input

```
In [56]: runfile('C:/Users/M.Hamzah Iqbal/Task1_Lab3.py', wdir='C:/Users/M.Hamzah Iqbal')
5
110
220
330
440
550
1650
```

Figure 1 Output

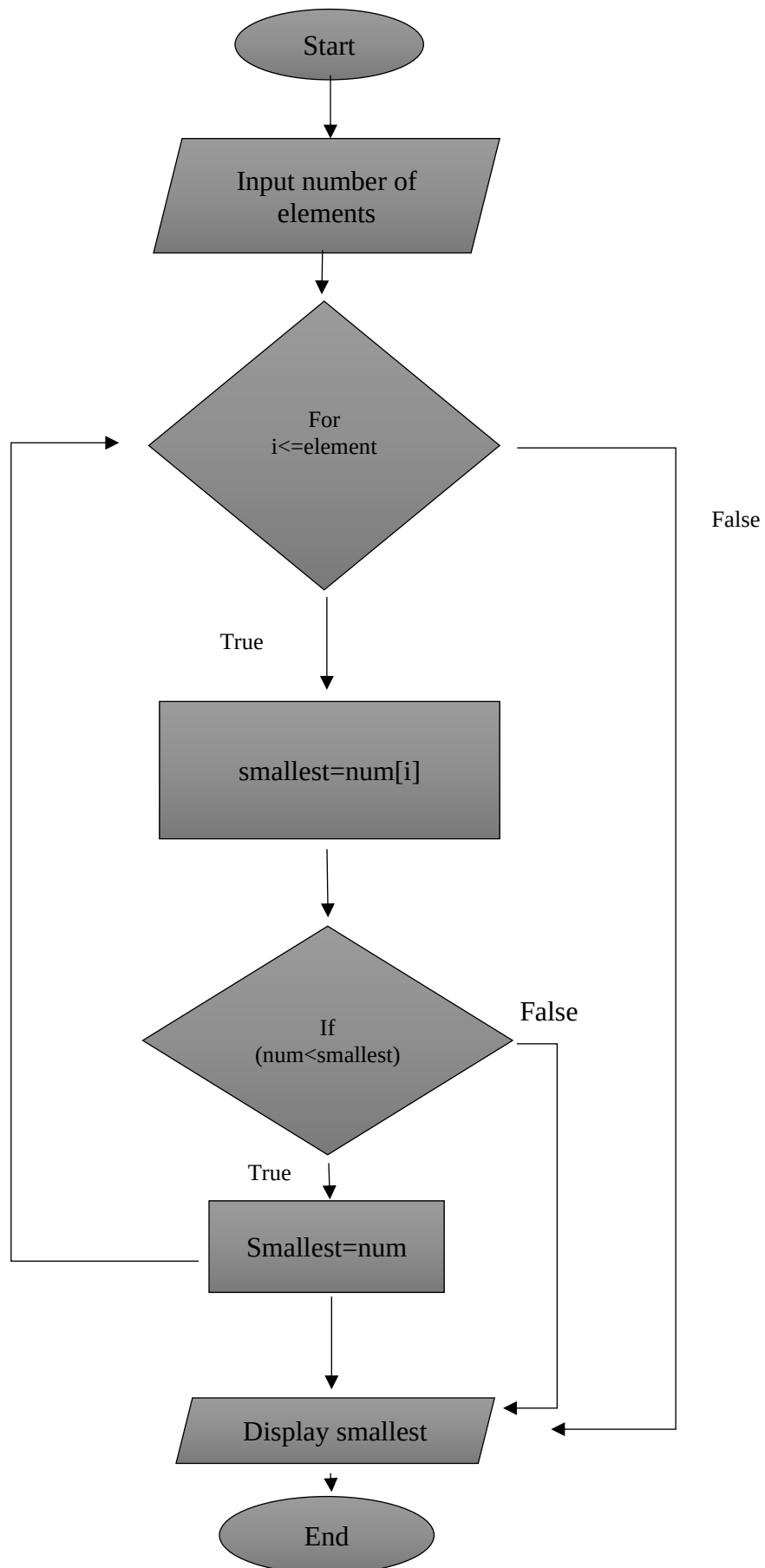
Task 02:

Write a python program to find the largest number among the three user input numbers.

Algorithm:

- Start
- Input: Read the first integer n (the number of values to be compared).
- Initialize: Set smallest to a very large value (e.g., infinity).
- Loop: For i from 1 to n:
 - a. Input: Read the next integer value.
 - b. Check: If value is less than smallest, then:
 - c. Update smallest to value.
- Output: Print smallest.
- End

Flowchart



Experimental result

```
1 #-*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5
6 num_count = int(input("Enter the number of values: "))
7 if num_count <= 0:
8     print("Invalid input. The number of values must be greater than zero.")
9 else:
10     smallest = int(input("Enter a number: "))
11     for _ in range(num_count - 1):
12         num = int(input("Enter a number: "))
13         if num < smallest:
14             smallest = num
15
16 print("The smallest number is:", smallest)
```

Figure 3 Input

```
In [55]: runfile('C:/Users/M.Hamzah Iqbal/Or
Task2_Lab3.py', wdir='C:/Users/M.Hamzah Iqbal')
Enter the number of values: 6
Enter a number: 12
Enter a number: 19
Enter a number: 45
Enter a number: 34
Enter a number: 2
Enter a number: 8
The smallest number is: 2
```

Figure 4 Output

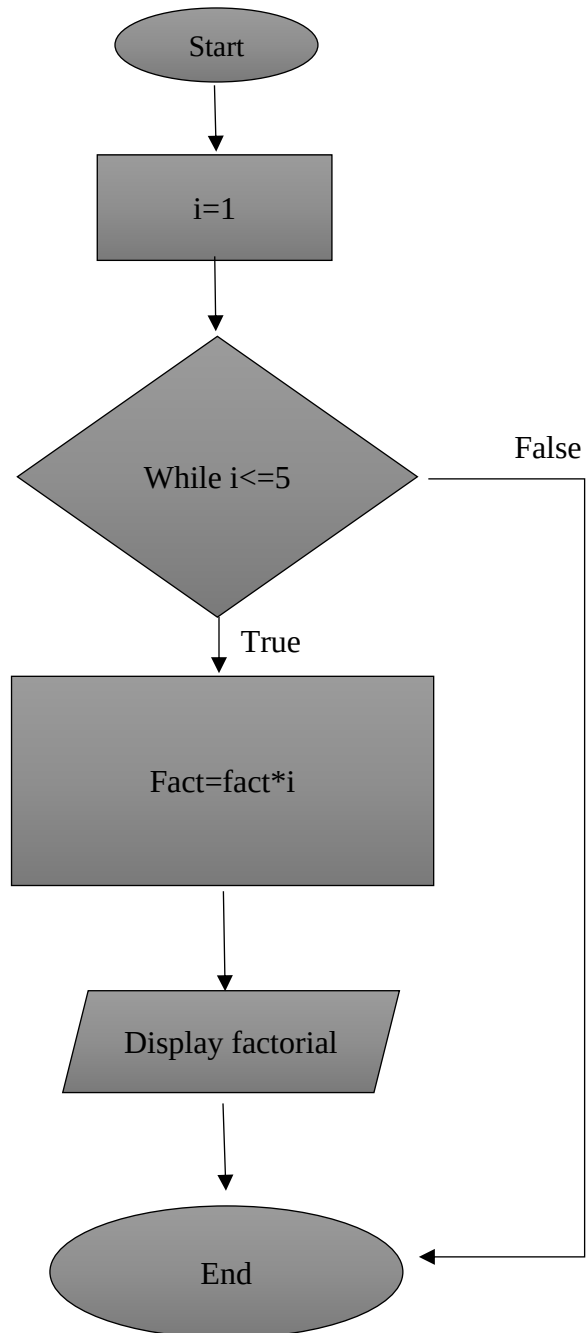
Task 03:

The factorial function is used frequently in probability problems. The factorial of a positive integer n (written $n!$ and pronounced “ n factorial”) is equal to the product of the positive integers from 1 to n . Using the factorial function, write a program that evaluates the factorials of the integers from 1 to 5. Print the results in tabular format as follows.

Algorithm

1. Start
2. Initialize: Set n to 5 (the maximum integer for which to calculate the factorial).
3. Print Header: Print the table header "Number" and "Factorial".
4. Loop: For each integer i from 1 to n :
 - a. Initialize: Set factorial to 1.
 - b. Inner Loop: For each integer j from 1 to i :
 - i. Update factorial to factorial * j .
 - c. Output: Print the value of i and its corresponding factorial.
5. End

Flowchart



Experimental result

```
1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5
6 i = 1
7 print(" X      Factorial of X")
8
9 while i <= 5:
10     fact = 1
11     j = 1
12
13     while j <= i:
14         fact = fact * j
15         j += 1
16
17     print(" ", i, "      ", fact)
18     i += 1
```

Figure 5 Input

```
In [54]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Python/Python
         wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Python/Python
         X      Factorial of X
         1      1
         2      2
         3      6
         4      24
         5      120
```

Figure 6 Output

Task 04

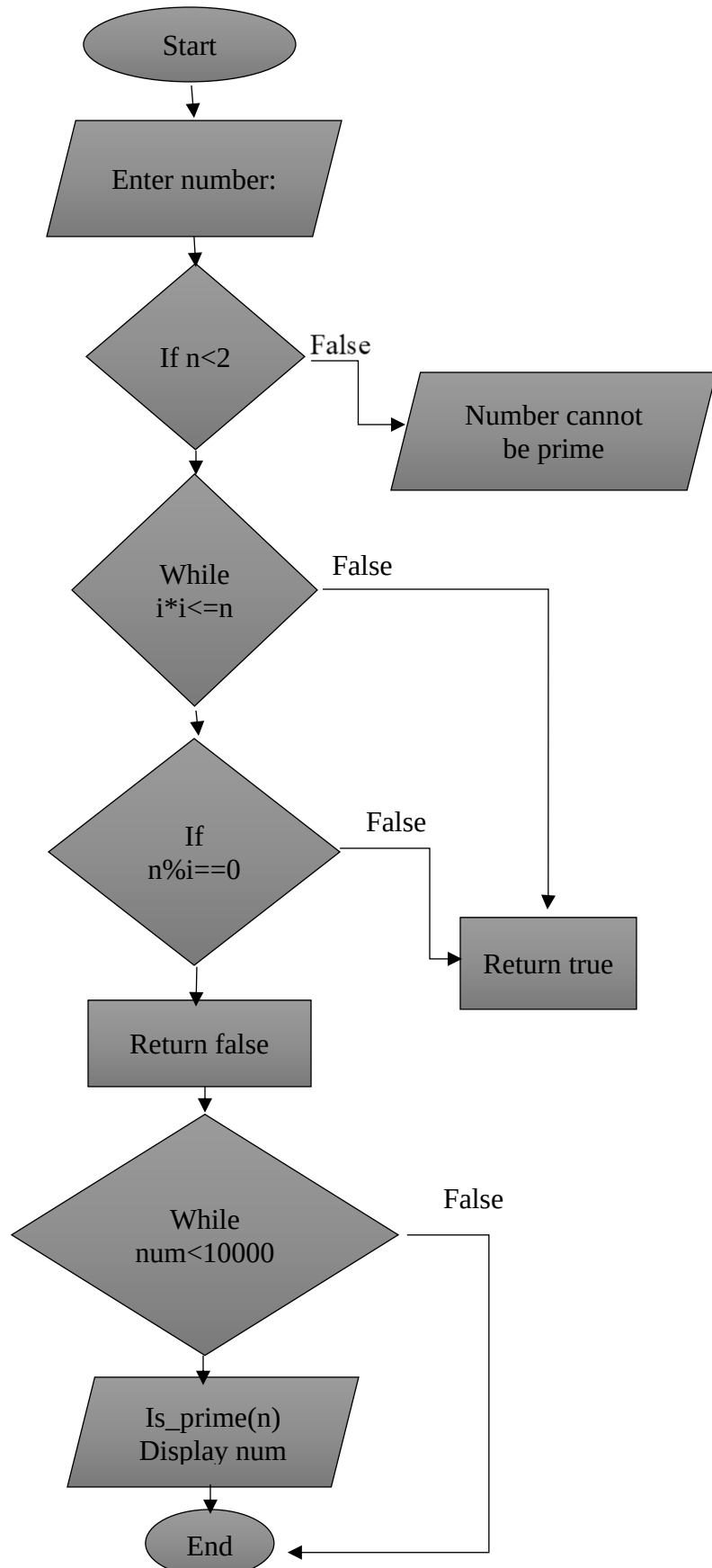
An integer is said to be prime if it is divisible only by 1 and itself. For example, 2, 3, 5 and 7 are prime, but 4, 6, 8 and 9 are not.

Write a function that determines if a number is prime. Use this function in a program that determines and prints all the prime numbers between 1 and 10,000.

Algorithm:

1. Start
2. Function Definition: Define a function `is_prime(num)` that:
 - a. Input: Accepts an integer num.
 - b. Check for Edge Cases:
 - If num is less than 2, return False (0 and 1 are not prime).
 - c. Loop: For each integer i from 2 to the square root of num:
 - i. If num is divisible by i, return False (not prime).
 - d. Return: If no divisors are found, return True (it is prime).
3. Main Program:
 - a. Print Header: Print "Prime numbers between 1 and 10,000".
 - b. Loop: For each integer i from 1 to 10,000:
 - i. Call `is_prime(i)`.
 - ii. If the function returns True, print i (it is a prime number).
4. End

Flowchart:



Experimental results

```
1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5
6
7 def is_prime(n):
8     if n < 2:
9         return False
10    i = 2
11    while i * i <= n:
12        if n % i == 0:
13            return False
14        i += 1
15    return True
16
17 print("Prime numbers between 1 and 10,000:")
18
19 num = 1
20 while num <= 10000:
21     if is_prime(num):
22         print(num, end=" ")
23     num += 1
```

Figure 7 Input

```
In [50]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Nutech_2_Sem/Computer programming Lab/Task_4.2_Lab3.py',
wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Nutech_2_Sem/Computer programming Lab')
Prime numbers between 1 and 10,000:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157
163 167 173 179 181 191 193 197 199 211 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317 331
337 347 349 353 359 367 373 379 383 389 397 401 409 419 421 431 433 439 443 449 457 461 463 467 479 487 491 499 503 509
521 523 541 547 557 563 569 571 577 587 593 599 601 607 613 617 619 631 641 643 647 653 659 661 673 677 683 691 701 709
719 727 733 739 743 751 757 761 769 773 787 797 809 811 821 823 827 829 839 853 857 859 863 877 881 883 887 907 911 919
929 937 941 947 953 967 971 977 983 991 997 1009 1013 1019 1021 1031 1033 1039 1049 1051 1061 1063 1069 1087 1091 1093
1097 1103 1109 1117 1123 1129 1151 1153 1163 1171 1181 1187 1193 1201 1213 1217 1223 1229 1231 1237 1249 1259 1277 1279
1283 1289 1291 1297 1301 1303 1307 1319 1321 1327 1361 1367 1373 1381 1399 1409 1423 1427 1429 1433 1439 1447 1451 1453
1459 1471 1481 1483 1487 1489 1493 1499 1511 1523 1531 1543 1549 1553 1559 1567 1571 1579 1583 1597 1601 1607 1609 1613
1619 1621 1627 1637 1657 1663 1667 1669 1693 1697 1699 1709 1721 1723 1733 1741 1747 1753 1759 1777 1783 1787 1789 1801
1811 1823 1831 1847 1861 1867 1871 1873 1877 1879 1889 1901 1907 1913 1931 1933 1949 1951 1973 1979 1987 1993 1997 1999
2003 2011 2017 2027 2029 2039 2053 2063 2069 2081 2083 2087 2089 2099 2111 2113 2129 2131 2137 2141 2143 2153 2161 2179
2203 2207 2213 2221 2237 2239 2243 2251 2267 2269 2273 2281 2287 2293 2297 2309 2311 2333 2339 2341 2347 2351 2357 2371
2377 2381 2383 2389 2393 2399 2411 2417 2423 2437 2441 2447 2459 2467 2473 2477 2503 2521 2531 2539 2543 2549 2551 2557
2579 2591 2593 2609 2617 2621 2633 2647 2657 2663 2671 2677 2683 2687 2689 2693 2699 2707 2711 2713 2719 2729 2731
2741 2749 2753 2767 2777 2789 2791 2797 2801 2803 2819 2833 2837 2843 2851 2857 2861 2879 2887 2897 2903 2909 2917 2927
2939 2953 2957 2963 2969 2971 2999 3001 3011 3019 3023 3037 3041 3049 3061 3067 3079 3083 3089 3109 3119 3121 3137 3163
3167 3169 3181 3187 3191 3203 3209 3217 3221 3229 3251 3253 3257 3259 3271 3299 3301 3307 3313 3319 3323 3329 3331 3343
3347 3359 3361 3371 3373 3389 3391 3407 3413 3433 3449 3457 3461 3463 3467 3469 3491 3499 3511 3517 3527 3529 3533 3539
3541 3547 3557 3559 3571 3581 3583 3593 3607 3613 3617 3623 3631 3637 3643 3659 3671 3673 3677 3691 3697 3701 3709 3719
3727 3733 3739 3761 3767 3769 3779 3793 3797 3803 3821 3823 3833 3847 3851 3853 3863 3877 3881 3889 3907 3911 3917 3919
3923 3929 3931 3943 3947 3967 3989 4001 4003 4007 4013 4019 4021 4027 4049 4051 4057 4073 4079 4091 4093 4099 4111 4127
4129 4133 4139 4153 4157 4159 4177 4201 4211 4217 4219 4229 4231 4241 4243 4253 4259 4261 4271 4273 4283 4289 4297 4327
4337 4339 4349 4357 4363 4373 4391 4397 4409 4421 4423 4441 4447 4451 4457 4463 4481 4483 4493 4507 4513 4517 4519 4523
4547 4549 4561 4567 4583 4591 4597 4603 4621 4637 4639 4643 4649 4651 4657 4663 4673 4679 4691 4703 4721 4723 4729 4733
4751 4759 4783 4787 4789 4793 4799 4801 4813 4817 4831 4861 4871 4877 4889 4903 4909 4919 4931 4933 4937 4943 4951 4957
4967 4969 4973 4987 4993 4999 5003 5009 5011 5021 5023 5039 5051 5059 5077 5081 5087 5099 5101 5107 5113 5119 5147 5153
5167 5171 5179 5189 5197 5209 5227 5231 5233 5237 5261 5273 5279 5281 5297 5303 5309 5323 5333 5347 5351 5381 5387 5393
5399 5407 5413 5417 5419 5431 5437 5441 5443 5449 5471 5477 5479 5483 5501 5503 5507 5519 5521 5527 5531 5557 5563 5569
5573 5581 5591 5623 5639 5641 5647 5651 5653 5657 5659 5669 5683 5689 5693 5701 5711 5717 5737 5741 5743 5749 5779 5783
5791 5801 5807 5813 5821 5827 5839 5843 5849 5851 5857 5861 5867 5869 5879 5881 5897 5903 5923 5927 5939 5953 5981 5987
6007 6011 6029 6037 6043 6047 6053 6067 6073 6079 6089 6091 6101 6113 6121 6131 6133 6143 6151 6163 6173 6197 6199 6203
6211 6217 6221 6229 6247 6257 6263 6269 6271 6277 6287 6299 6301 6311 6317 6323 6329 6337 6343 6353 6359 6361 6367 6373
6379 6389 6397 6421 6427 6449 6451 6469 6473 6481 6491 6521 6529 6547 6551 6553 6563 6569 6571 6577 6581 6599 6607 6619
6637 6653 6659 6661 6673 6679 6689 6691 6701 6703 6709 6719 6733 6737 6761 6763 6779 6781 6791 6793 6803 6823 6827 6829
```

Figure 8 Output

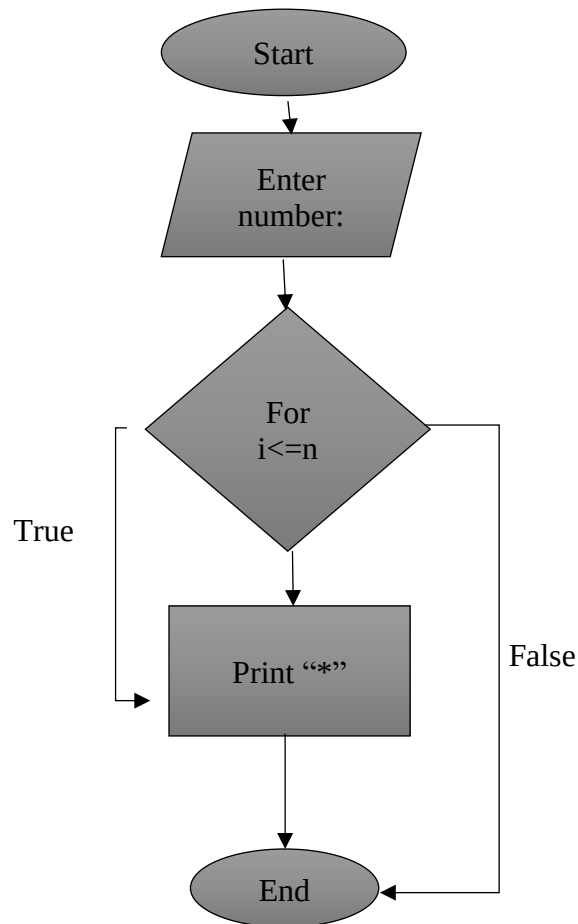
Task 05:

Write a program that reads a number (Between 1 and 30). Your program should print a line containing that number of adjacent asterisks. For example, if your program reads the number Seven, it should print `*****`.

Algorithm:

1. Start
2. Input: Read an integer n (the number of asterisks to print).
3. Check Validity:
 - If n is less than 1 or greater than 30, print an error message and terminate the program.
4. Print Asterisks: Print a line containing n adjacent asterisks.
5. End

Flowchart



Experimental Results

```
3 @author: M.Hamzah Iqbal
4 """
5
6 num = int(input("Enter a number: "))
7 if num > 30:
8     num = int(input("Sorry, enter a number again that is from 1-30: "))
9
10 if 1 <= num <= 30:
11     for _ in range(num):
12         print("*", end="")
13     print()
14 else:
15     print("Invalid input. Please enter a number between 1 and 30.")
```

Figure 9 Input

```
In [59]: runfile('C:/Users/M.Hamzah Iqbal/untitled16.py', wdir='C:/Users/M.Hamzah Iqbal')
Enter a number: 8
*****
```

Figure 10 Output

For loop tasks

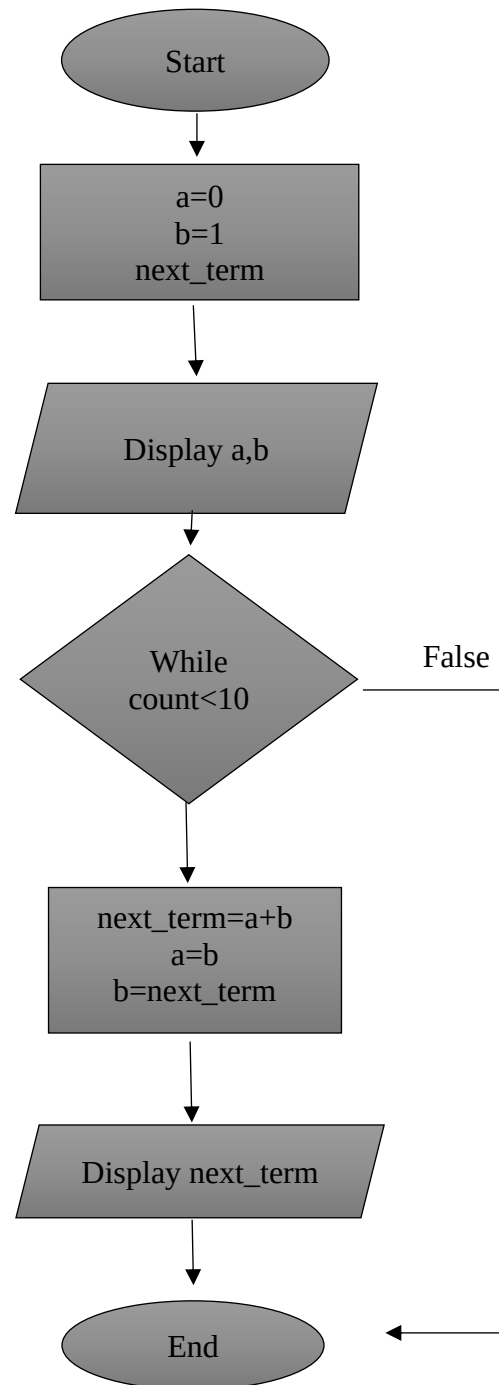
1. Implement Fibonacci series up to 10 terms. Your code must print this output: 0 1 1 2 3 5 8 13 21 34

Algorithm:

- Start
- Initialize:
- Set a to 0 (first term).
- Set b to 1 (second term).
- Set count to 2 (to track the number of terms printed).
- Print: Print a (the first term).
- Print: Print b (the second term).
- Loop: While count is less than 10:
 - Calculate: Set next_term to a + b.

- Print: Print next_term.
- Update: Set a to b (move to the next term).
- Update: Set b to next_term (move to the next term).
- Increment: Increment count by 1.
- End

Flowchart



Experimental results:

```
1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5
6 def fibonacci_series(n):
7     a, b = 0, 1
8     for _ in range(n):
9         print(a, end=' ')
10        a, b = b, a + b
11
12 fibonacci_series(10)
```

Figure 11 Input

```
In [46]: runfile('C:/Users/M.Hamzah Iqbal,
Task1_forLoop.py', wdir='C:/Users/M.Hamza
0 1 1 2 3 5 8 13 21 34
```

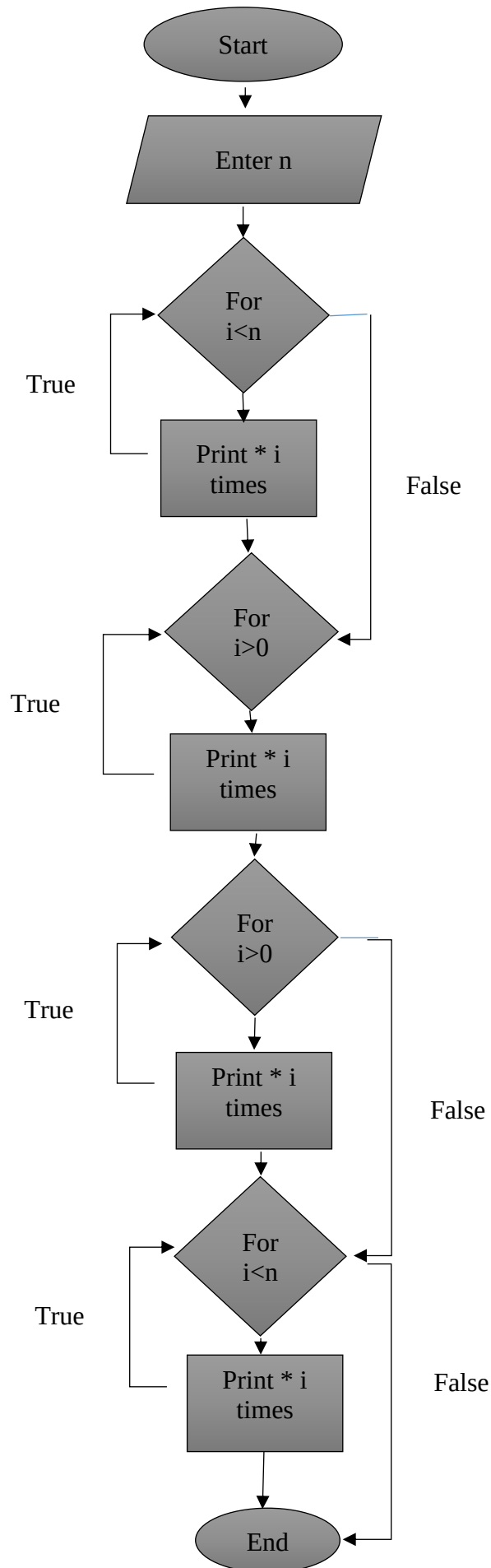
Figure 12 Output

2. Write a program that uses for statements to print the following patterns separately. All asterisks (*) should be printed by a single statement of the form print (*).

Algorithm

1. **Start**
2. **Pattern (a):** Print a left-aligned right-angled triangle.
 - a. Loop from 1 to **n** (where **n** is the number of rows).
 - b. For each row, print **i** asterisks using a single print statement.
3. **Pattern (b):** Print a left-aligned inverted right-angled triangle.
 - a. Loop from **n** down to 1.
 - b. For each row, print **i** asterisks using a single print statement.
4. **Pattern (c):** Print a right-aligned inverted right-angled triangle.
 - a. Loop from **n** down to 1.
 - b. For each row, print spaces followed by **i** asterisks using a single print statement.
5. **Pattern (d):** Print a right-aligned right-angled triangle.
 - a. Loop from 1 to **n**.
 - b. For each row, print spaces followed by **i** asterisks using a single print statement.
6. **End**

Flowchart :



Experimental results

```

1 # -*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5
6 def print_patterns():
7     rows = 10
8     diamond_rows = 5
9     spacing = 12
10
11     print("(a)", " " * (spacing - 3), "(b)", " " * (spacing - 3), "(c)", " " * (spacing - 3), "(d)")
12
13     for i in range(rows):
14         for j in range(i + 1):
15             print('*', end='')
16         for space in range(spacing - i):
17             print(' ', end='')
18
19         for j in range(rows - i):
20             print('*', end='')
21         for space in range(spacing - (rows - i)):
22             print(' ', end='')
23
24         for space in range(i):
25             print(' ', end='')
26         for j in range(rows - i):
27             print('*', end='')
28         for space in range(spacing - (rows - i)):
29             print(' ', end='')
30
31         for space in range(rows - (i + 1)):
32             print(' ', end='')
33         for j in range(i + 1):
34             print('*', end='')
35
36     print()
37
38     print()
39
40     for i in range(2 * diamond_rows - 1):
41         spaces = abs(diamond_rows - 1 - i)
42         stars = 2 * (diamond_rows - spaces) - 1
43         print(' ' * (spacing * 2), end='')
44         print(' ' * spaces + '*' * stars)
45
46 print_patterns()
47

```

Figure 13 Input

```
In [40]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Nutech_2_Sem/Computer programming Lab/untitled15.py', wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/Nutech_2_Sem/Computer programming Lab')
```

```
(a)          (b)          (c)          (d)
```

```
*            *            *            *
```

```
**           *            **           *
```

```
***          *            ***          *
```

```
****         *            ****         *
```

```
*****       *            *****      *
```

```
*****       *            *****      *
```

```
*****       *            *****      *
```

```
*****       *            *****      *
```

```
*****       *            *****      *
```

```
*****       *            *****      *
```

```
*****       *            *****      *
```

```
*****       *            *****      *
```

```
*****       *            *****      *
```

```
          *
```

```
        **
```

```
      ****
```

```
    *****
```

```
  *****
```

```
*****
```

```
*****
```

```
  ****
```

```
***
```

```
*
```

Figure 14 Output

Discussion of Results

- The lab successfully demonstrated the use of for and while loops in Python.
- Various loop structures were tested, including iteration over lists, numeric sequences, and conditional looping.
- The Fibonacci sequence was correctly generated using a for loop.
- The factorial calculation was successfully implemented using loops.
- The prime number identification function efficiently filtered prime numbers within the given range

Conclusions

- Loops are essential for iteration in programming and help in automating repetitive tasks.
- The while loop is useful when the number of iterations is unknown, while the for loop is more suited for fixed iterations.
- Proper loop termination conditions are necessary to avoid infinite loops.
- Nested loops and loop control statements (break, continue) provide flexibility in problem-solving